Rotaract Club of Xiamen University Malaysia | Rotary Club of Kuala Lumpur Dynamics | Rotary Club of Tun Razak Exchange

SDC Hackathon

Technical Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

Instagram: @sdc_xmumhack ***Email:*** sdc.xmumhack@gmail.com

SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: [Insert date]

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	Array Five
Project / Solution Name	SCDC Recovery
Name (Leader)	Yosef Kristian Tirta
Email & Phone (Leader)	MBT2309423@xmu.edu.my & **+60 1126618368**
University Affiliation(s)	Xiamen University Malaysia
Live Prototype URL	https://array-five.replit.app
Code Repository URL (public / access granted to judges)	https://github.com/YosefKT1604/ARRAY-FIVE

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

The system is a layered web application deployed on Replit. Business owners use a React frontend to receive disruption reports (e.g. a simulated supplier WhatsApp message), review their impact and approve recovery plans. The frontend calls a FastAPI backend through a REST API. The backend validates every request, checks user permissions, stores records in a SQLite database, and coordinates three modules:

- AI module: turns supplier messages into structured incidents and writes plain-language explanations using the optional Google Gemini API (Gemini 3.1 Flash-Lite via LiteLLM), with rule-based fallback.
- Impact analysis: follows supplier → delivery → resource → job → commitment dependencies to find affected commitments and resource shortfalls.
- Recovery planning: generates candidate plans, validates them with deterministic rules (stock, lead time, deadlines, double-booking) and ranks the feasible ones.

End-to-end flow: disruption reported → AI extraction (optional) → validation and storage → impact analysis → plan generation and validation → ranking and explanation → owner approval → drafted messages → activity log.

AI never decides feasibility or writes to the database. Unconfirmed facts are flagged, edited plans are revalidated, and only the owner can approve. Code is managed on GitHub.

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
Python 3.12	Main programming language for the backend using FastAPI and the algorithm modules (impact analysis, recovery planning, AI integration).	Python Software Foundation (PSF) License
FastAPI 0.153.0	Backend web framework providing the REST API and serving the website	MIT
Uvicorn 0.54.0	Web server that runs the FastAPI application	BSD 3-Clause
Pydantic 2.14.1	Data validation and shared data contracts; rejects invalid incidents, plans and results	MIT
SQLAlchemy 2.1.5	Database access layer between the backend and the database	MIT
LiteLLM 1.100.5	Connects the AI module to the Google Gemini API through a replaceable adapter	MIT
pytest 9.1.1	Automated backend testing (584 tests, including 42 recovery-planning tests)	MIT
React 19.3.1	Frontend user interface library	MIT
React Router 7.18.4	Page navigation in the frontend	MIT
TanStack Query 5.104.5	Fetching and caching data from the backend API	MIT
Tailwind CSS 4.3.3	Styling of the user interface	MIT
Vite 8.3.5	Frontend build tool and development server	MIT
Vitest 5.0.5	Frontend unit and component testing	MIT

Name & Version	Purpose in Project	License
Playwright 1.64.1	End-to-end testing of the full demo journey	Apache License 2.0
Mock Service Worker 3.0.3	Mock API data for frontend development and tests	MIT
TypeScript 5.9.3	Main language for the frontend	Apache License 2.0
SQLite 3	Database for business data, plans and the activity log	Public Domain

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
Google Gemini API (Google): Gemini 3.1 Flash-Lite, called through the LiteLLM library	Reads supplier messages into structured incident details, writes plain-language explanations, and polishes drafted supplier and customer messages. Every output is checked against business data, and the app runs without it using rules and templates.	Free

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
GitHub (GitHub, Inc.)	Code repository, version control and team collaboration	Free plan / Global
Replit Deployments (Replit)	Hosts the live prototype: one deployment runs the FastAPI backend and serves the React website	Autoscale deployment, Replit Core plan / Region managed by Replit
Replit Secrets (Replit)	Stores the Gemini API key and AI settings outside the code	Included with Replit

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
Gemini 3.1 Flash-Lite	Google	Pre-trained, used through an API	Gemini API Additional Terms of Service and Google APIs Terms of Service

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
Demo business data (seed.json)	github.com/YosefKT1604/ARRAY-FIVE: fixtures/demo/	Synthetic	Apache 2.0	N
Demo incident and expected results (incident.json, expected_impact.json, expected_plans.json)	Same repository: fixtures/demo/	Synthetic	Apache 2.0	N
AI extraction evaluation set (24 supplier messages)	Same repository: fixtures/ai/extraction_cases.json	Synthetic	Apache 2.0	N

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
AI explanation evaluation set (20 cases)	Same repository: fixtures/ai/explanation_cases.json	Synthetic	Apache 2.0	N
AI draft-message evaluation set (18 cases	Same repository: fixtures/ai/draft_cases.json	Synthetic	Apache 2.0	N
Frontend demo scenarios	Same repository: frontend/src/mocks/scenarios/	Synthetic	Apache 2.0	N
Recovery-planning test scenarios (42 tests)	Same repository: tests/recovery/	Synthetic	Apache 2.0	N

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ Yes ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
Claude Code (Anthropic), model Claude Sonnet 5.5	P1 (Frontend): helped build the React screens, connect them to the backend API, and debug UI issues.	Major
Claude Code (Anthropic), model Claude Opus 5.5	P2 (Backend): helped build the FastAPI routes, database models, permission checks and activity log, write tests, and debug.	Major
Claude Code (Anthropic), model Claude Opus 5.5	P3 (Impact analysis): helped build the dependency-graph impact engine and its tests, and debug.	Major
Claude Code (Anthropic), model Claude Opus 5.5	P4 (Recovery planning): generated the plan generator, deterministic validator, ranking logic and 42 tests; debugged chair-count, ranking and confirmation issues and added new plan types; checked integration with the team's impact engine.	Major
Claude Code (Anthropic), model Claude Opus 5.5	P5 (AI module): helped build the Gemini adapter, message extraction, explanations and drafts, the evaluation sets, and debug.	Major

Tool & Version	Used For	Extent of Use (minor / moderate / major)
Claude (Anthropic)	P1/team lead: project planning, integration guidance, code review and live testing, pitch script and slide/document checks	Major
OpenAI Codex	Used by all members (P1–P5) to learn backend engineering concepts and Python syntax	Minor

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

Each member owned and reviewed the AI-assisted code for their module before merging. Work was done on separate branches and merged through pull requests. AI-generated code was verified with automated tests: 584 backend tests (including 42 recovery-planning tests) and 78 frontend tests, covering failure cases such as late suppliers, insufficient stock and invalid edits. Demo results were checked against agreed expected outputs, and the live app was tested end to end. The AI module's safety checks were evaluated on 62 synthetic cases. Team rules forbid secrets in code and showing any plan as feasible without deterministic validation.

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☑ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
Replit	Platform	Hosts the live prototype: one deployment runs the FastAPI backend and serves the React website; Replit Secrets stores the Gemini API key
Alpha Red	Mentoring	Reviewed system architecture and validation approach

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

Neither resource was essential to the core functionality, and both could be replaced. Replit hosts the live prototype and stores its API key securely, but the application is standard Python (FastAPI) and React with open-source libraries, so it also runs locally or on any other hosting platform without changes. The core logic (disruption analysis, recovery-plan generation and deterministic validation) does not depend on Replit. Alpha Red's mentoring guided our design decisions but is not part of the running system.

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

The prototype collects no real personal data. It uses only synthetic demo data for a fictional business: invented supplier, customer and staff names, simulated supplier messages, business records, recovery plans, approval decisions and an activity log, stored in a SQLite database inside the Replit deployment.

If the optional AI feature is enabled, only the supplier message text and validated analysis results are sent to Google's Gemini API (Gemini 3.1 Flash-Lite). No contact details, passwords or payment information are sent. The free tier's terms allow Google to use submitted content to improve its services, so only synthetic data is used. The API key is stored in Replit Secrets, never in the code or repository.

The AI cannot modify stored records, and only the owner role can approve plans. Login is simulated with demo accounts. Demo data is reset whenever the database is reseeded.

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

Hallucination: the model may invent suppliers, quantities or dates. Mitigation: output must match a strict schema and is checked against business records; every plan passes a deterministic validator, and invented entities are rejected.

Misreading messages: informal or mixed-language messages (e.g. "Friday", "kerusi") can be ambiguous. Mitigation: AI and rule-based readings are compared; disagreements and uncertain fields are flagged for the user to confirm.

Single provider (Google Gemini), outages and rate limits (the free tier limits requests). Mitigation: a replaceable LiteLLM adapter lets another model be swapped in by configuration, and rule-based fallback keeps the full workflow running without AI.

Simplified planning model: e.g. rental return terms are not recorded. Mitigation: unknown facts are marked “needs confirmation”, never “feasible”.

Over-trust: Mitigation: AI text is labelled, AI never decides feasibility or edits records, and the owner approves every plan.

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

- ☑ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☑ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☑ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.

- ☑ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☑ Any use of official SDC sponsor resources has been declared in Section 5.
- ☑ The prototype described here is accessible to judges through the links provided in Section 1.
- ☑ All fields in this template have been completed within the word limits stated.

8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.

[Signature]



[Date]
24 October 2026